Informe técnico de la Entrega 2

La implementación traduce todos los tipos de construcción válidos del AST a TAC tipado y ubicado en el código fuente, mediante un único núcleo compartido por la CLI y el IDE, con disposiciones estáticas de símbolos extendidos. T9/T10 resolvieron las brechas iniciales de fuentes aceptadas, entre ellas print, residuo, ternarios perezosos, valores de asignación, almacenamiento de undefined/null, valores sin tipo y try/catch. Este informe relaciona las cuatro páginas del enunciado con los contratos implementados y la evidencia ejecutable; no constituye una prueba de corrección exhaustiva de todos los programas, una calificación futura ni una certificación del navegador.

Fundamento y resultado

Referencia normativa: instrucciones canónicas, una transcripción completa en español de las páginas 1–4 del `Proyecto_2__Compiladores.pdf` proporcionado por el equipo, incluidos los límites de página y la rúbrica. El original permanece local, ignorado y sin cambios; su SHA-256 es `0ba21279ad758dd8ef69344adae9d31193d5ca4d064ce525257da6974c707bdc`. Una copia nueva del repositorio Git incluye la transcripción y no requiere el PDF original. La matriz siguiente conserva citas breves en español; los números de página se refieren a las páginas del PDF original, no a números de línea del repositorio.

El resultado principal es un único compilador con responsabilidades explícitas, operaciones de ejecución protegidas con guardas y rechazo atómico ante diagnósticos reales de compilación. La arquitectura explica las decisiones; el diseño del TAC especifica el IR implementado, los ejemplos y los supuestos. Las instrucciones de ejecución y la aceptación proporcionan los puntos de entrada ejecutables. Esta actualización final sigue a las publicaciones de T9 `9ce55ce` y T10 `633aad1`, incluida la verificación independiente y la corrección del contrato de campos de T10. Esta actualización no identifica ninguna nueva brecha normativa concreta de implementación; las comprobaciones humanas y de evaluación pendientes se enumeran más abajo.

Matriz de requisitos del PDF y evidencia

Los archivos del compilador mencionados a continuación están en `internal/compiscript`; los archivos de traducción (lowering) están en su subdirectorio `tac`. Las rutas que comienzan con `cmd`, `testdata` o `web` son relativas a la raíz del repositorio. Las pruebas son puntos de entrada concretos de evidencia, no una afirmación de cobertura de líneas medida. “Implementado” significa que la capacidad nombrada tiene implementación y pruebas ejecutables; no es una garantía para todo programa fuente posible.

Página / requisito textual breve	Implementación y evidencia	Resultado
1: “traducen cada construcción de Compiscript”	Suites T9/T10 del núcleo y de la CLI construida, y las pruebas conservadas `independent_t9_verifier_test.go`, `independent_t10_verifier_test.go`, `independent_t10_b1_recheck_test.go`; inventario de tipos del AST	Se despachan los 18 tipos válidos de sentencia y los 14 de expresión; brecha inicial resuelta, se conservan guardas estáticas y de ejecución explícitas
1: “asignación, operaciones binarias y unarias, copia”	`tac/ir.go`, `expression.go`, `values_effects_test.go`, `dynamic_exceptions_test.go`; traducción numérica en la referencia	Implementadas las operaciones tipadas/protegidas, el residuo y las asignaciones que producen valor
1: “saltos condicionales e incondicionales, etiquetas”	`tac/flow.go`, `loops.go`, `switch.go`; `conditions_test.go`, `loops_test.go`, `switch_test.go`; ejemplos versionados	Implementado
1: “llamadas a procedimientos, acceso indexado”	`tac/calls.go`, `collections.go`; `functions_test.go`, `collections_test.go`	Implementado
1: “consistente y estar documentada”	Referencia de instrucciones, texto/JSON tipado, pruebas de formato `tac/ir_test.go`, `procedures_test.go`	Sintaxis implementada documentada
1: “(+, -, *, /) respetando precedencia y asociatividad”	`tac/expression.go`, `TestCompileNumeric`; `numeric.cps`, `i2f` explícito y división de punto flotante	Aritmética nombrada implementada
1: “(&&, ||, !) y de comparación (==, !=, <, <=, >, >=)”	`tac/flow.go`, `conditions_test.go`; comprobaciones de ejecución en cortocircuito y de trampas (traps)	Lógica y comparaciones nombradas implementadas

Página / requisito textual breve	Implementación y evidencia	Resultado
1: “declaraciones y asignaciones de variables y constantes”	`tac/lower.go`; `compile_test.go`, pruebas semánticas de const, `TestCollectionAtomicRejection`, suites de valores/efectos y de valores dinámicos/excepciones	Declaraciones tipadas y desconocidas como undefined; sin valores por defecto inventados; las guardas de ejecución preservan los contratos de const y de tipo
2: “if/else, while, do-while, for y switch”	`flow.go`, `loops.go`, `switch.go`; archivos de ejemplo nombrados en `testdata/compiscript/tac`, `TestAcceptanceValidLoops`	Implementado dentro de los contratos semánticos actuales (switch booleano)
2: “break y continue”	Pila de transferencias compartida; pruebas de ciclos, switch y colecciones, `acceptance.cps` mixto	Implementados hacia el destino elegible más cercano
2: “paso de parámetros, llamadas y valores de retorno”	`tac/functions.go`, `calls.go`, `frames.go`, `dynamic.go`; suites de marcos y de valores dinámicos/excepciones	Parámetros formales tipados y sin tipo con guardas de ejecución reales; la ausencia de anotación de resultado sigue produciendo null
2: “funciones recursivas”	Dependencias por punto fijo y marcos nuevos; `recursion_test.go`, `recursion_dependencies_test.go`, `recursion.cps`	Recursión directa y mutua implementada; no se promete terminación
2: “funciones anidadas y closures”	`tac/closures.go`; `TestClosureExecution`, `closure_compositions_test.go`, `closures.cps`	Implementadas celdas compartidas retenidas y alias que escapan
2: “creación de objetos e invocación del constructor”	`tac/objects.go`; `TestObjectExecution`, `TestObjectThreeLevelPolicyA`, `objects.cps`	Implementada la política explícita de inicialización A
2: “acceso a atributos” / “incluyendo el uso de this”	Lecturas/escrituras de campos y métodos enlazados; `objects_test.go`, `independent_r3_test.go`, regresiones del contrato de campos de T10	Receptores nominales/heredados y acceso dinámico verificado; los campos sin anotación siguen siendo desconocidos y los anotados permanecen restringidos
2: “creación de listas” / “mediante índices”	`tac/collections.go`; `TestCollectionExecution`, pruebas conservadas de T3/T9/T10, `collections.cps`	Listas vacías, anidadas, irregulares y null, lecturas/escrituras y vistas desconocidas verificadas; además se admite foreach
2: “Algoritmo de asignación de temporales”	`tac/temps.go`; `tac/temps_test.go`, `TestCompileRepeatedExpressions`	Ranura libre más pequeña, conjuntos independientes por cinta
2: “Reciclaje de temporales una vez que su valor ha sido consumido”	Liberación en el consumidor final; pruebas de rango de vida y reutilización en `tac/temps_test.go`; contrato de tiempos de vida	Implementado, sin afirmar optimalidad
2: “direcciones de memoria o desplazamientos, tamaños, etiquetas”	`tac/symbols.go`; `TestCompilationSymbols`, `tac/symbols_test.go`, `symbols_cli_test.go`	Desplazamientos y tamaños deterministas en celdas abstractas y etiquetas reales, no direcciones físicas
2: “parámetros, variables locales, temporales, valor y dirección de retorno, enlaces de control y de acceso”	`tac/frames.go`, disposiciones de entornos conservadas; `TestFunctionFrameMetadata`, `TestCompilationSymbols`	Celdas explícitas de control y resultado del marco, y enlaces léxicos; referencia de disposición
2: “interactuar con cada fase de la compilación”	Ubicaciones AST/fuente -> identidades/ámbitos semánticos -> Generate -> ubicaciones de codegen; comprobaciones de inmutabilidad en `TestAcceptanceMixedExecution`	Mismo inventario de identidades, reporte de compilación separado que no modifica las fases anteriores
2: “extensión .cps” / “Código intermedio (TAC)” / “Reporte de errores” / “Estado de la tabla de símbolos”	Comprobaciones de extensión y UTF-8 en la CLI; reporte con `tac`, `diagnostics`, `scopes` y `codegen` opcional; `cmd/compiscript-tac/main_test.go`, pruebas de símbolos de la CLI	Implementados texto/JSON y vistas del IDE
3 paso 1: “carga o escribe un archivo .cps a través del IDE”	Controles de carga/edición/guardado y compilación en `web`; pruebas de Node de carga de archivos y de solicitudes obsoletas; guía del IDE	Conexiones probadas; revisión manual en navegador pendiente

Página / requisito textual breve	Implementación y evidencia	Resultado
3 pasos 2–4: “(ANTLR) construye el árbol sintáctico” / “validando las reglas” / “Si no existen errores”	`frontend.Parse` -> `semantic.Analyze` -> `tac.Generate` en Compile; pruebas de paridad de la fachada y de recuperación atómica	Implementado una sola vez por compilación; no hay un segundo compilador
3 paso 5: “errores encontrados con su tipo y ubicación”	Fase, código y ubicaciones tipados; representación en la CLI y `tacText` por HTTP; `TestMixedAcceptanceAPI`, pruebas de Node de representación de diagnósticos	Implementado; los errores de lenguaje con HTTP 200 se distinguen de los 4xx de transporte
3 paso 6 y entregable: “batería de pruebas” / “casos exitosos y fallidos”	Batería ejecutable, pruebas públicas de CLI/API y ejecución acotada con oráculo	Comprobaciones focalizadas aprobadas; la disponibilidad el día de la evaluación sigue siendo responsabilidad humana
3: “repositorio de GitHub, con commits individuales”	Enlace al repositorio en la raíz e historial de commits auditado más abajo	Registros locales de autoría establecidos; pendientes la constancia de contribuciones reales y el acceso remoto/de evaluación
3: “Documentación de la arquitectura” / “cómo ejecutar el compilador”	Arquitectura, guía de ejecución e índice raíz	Markdown versionado entregado
3: “lenguaje intermedio diseñado, con ejemplos de traducción y los supuestos”	Referencia TAC: traducción numérica, marcos, closures, objetos/listas, valores/efectos, valores protegidos y manejadores	Entregado con supuestos explícitos de inicialización, formato, fallos y desenrollado
3: “IDE funcional que permita escribir y compilar”	Endpoint HTTP del núcleo compartido más paneles de TAC, errores y símbolos; httptest de Go y 39 casos de Node	Frontera funcional probada; evaluación visual real pendiente
4: “funcionar correctamente el día de la presentación” / “documentado y organizado”	Índice de documentación y pruebas reproducibles; lista de preparación manual más abajo	No se pueden certificar las condiciones futuras de la presentación
4: “Diseño de CI” 25 / “Generación de TAC” 65 / “Tabla de Símbolos” 10	Referencia/diseño, batería ejecutable de generación y reporte de símbolos extendidos/disposición, respectivamente	Pesos de calificación que suman 100 puntos, **no porcentajes de avance**

Verificación observada y procedencia

Los registros de implementación proporcionados por el agente principal distinguen la prueba de cada unidad de las adiciones posteriores que solo agregan pruebas. T9 y la versión final de T10 reportan cada una suites completas de Go aisladas (18 paquetes con pruebas, cuatro sin pruebas), compilación, vet forzado y 39 casos de Node aprobados. La última adición de T10 sobre campos nominales, que **solo agrega pruebas**, se realizó después de su verificación con la suite completa; el agente principal volvió a ejecutar después los cuatro paquetes afectados, sin cambios en el código de producción. Esta actualización, que solo modifica documentación, no volvió a ejecutar la generación, la compilación ni vet completos, y no afirma un resultado nuevo de la suite completa posterior a `633aad1`.

La evidencia histórica incluye comparaciones exactas de compatibilidad: 45 del autor de T4 y 60 del verificador independiente, más nueve cambios de panic a rechazo atómico; 39 comparaciones de T6 que eliminan solo el campo aditivo autorizado `codegen` y 36 comprobaciones JSON repetidas; 37 comparaciones HTTP, siete DOM y 31 de CLI del autor de T7, con comprobaciones independientes de API e interfaz. Son conjuntos acotados que se superponen, **no un total general sumable**. La evidencia RED original de T7 es una transcripción conservada (API 404; Node con 14 fallos y cuatro aprobados), no un artefacto de registro sin procesar guardado por separado. La aprobación nativa histórica aplica solo a T1; la revisión local posterior en el clon estuvo deshabilitada o sin gestionar, no aprobada.

Unidad publicada	Implementación registrada por el agente principal y evidencia independiente
T9 `9ce55ce`	31 escenarios compartidos de núcleo/CLI y 82 comparaciones exactas con la línea base; de forma independiente, 20 casos de ejecución/salida, 30 comparaciones con archivos de prueba anteriores, 20 comparaciones de Analyze, diez rechazos atómicos y cuatro comprobaciones de política de la CLI. Verificador permanente conservado y cuatro paquetes reejecutados por el agente principal
T10 `633aad1`	69 escenarios compartidos de núcleo/CLI más comprobaciones de errores fatales; 170 comparaciones exactas finales (33 de texto/JSON/Analyze con archivos de prueba anteriores, 110 nuevas de Analyze, 27 de reportes rechazados) y 51 variantes de fuentes. Comprobación independiente más amplia: 29 composiciones aprobadas y 83 comparaciones exactas antes de B1; cinco fallos de campos desconocidos se corrigieron una vez y luego esos cinco más ocho regresiones directas pasaron en una reverificación limitada

La corrección B1 de T10 reemplaza el estrechamiento privado de campos null de T9: la ausencia de anotación implica almacenamiento de campo desconocido incluso cuando se inicializa con valores null o numéricos, mientras que los campos sin inicializador siguen siendo undefined. Los campos nominales con anotación explícita conservan sus guardas antes de los efectos. Ambos archivos independientes de T10 son pruebas permanentes sobre el núcleo actual y la CLI construida actual, no pruebas que requieran controles externos de línea base. Los conteos superpuestos de escenarios y comparaciones anteriores no deben sumarse como un total de cobertura.

La auditoría inicial de T8 en `f416fa2` observó correctamente ocho fuentes válidas para Analyze que se rechazaban durante la traducción, entre ellas `let x=1; print(1);` (`TAC_UNSUPPORTED` en 1:10). Esa es evidencia RED histórica de la brecha de requisitos, **no el comportamiento actual**. Sus comprobaciones de documentación originales cubrieron diez archivos de prueba y 20 invocaciones de la CLI; la actualización final que se describe a continuación cubre doce y 24.

La verificación nueva de la actualización final usó Go sin conexión, con HOME/TMPDIR/GOCACHE externos y una caché de módulos existente:

- Se compilaron los binarios de TAC y Analyze fuera del repositorio; se comprobaron la ayuda, los doce ejemplos TAC versionados en texto y JSON, las entradas rechazadas en texto y JSON, el código de salida ante un formato inválido y los códigos de salida del Analyze heredado. Las 24 invocaciones exitosas de ejemplos terminaron con código 0 y stderr vacío; los hashes de las fuentes no cambiaron. No se introdujo ni se invocó ningún servicio de ejecución del producto.
- `go test ./internal/compiscript ./cmd/ide -run '^Test(ValuesEffects|DynamicExceptions|IndependentT9|IndependentT10|MixedAcceptanceAPI)' -count=1`: ambos paquetes aprobaron, incluidas las regresiones independientes conservadas, los efectos de valores/salida/fallos, los límites de errores fatales de los manejadores y la paridad HTTP.
- Las expectativas del oráculo mixto siguen siendo `first=30`, `second=43`, `independent=7`, `continued=57`, `shared=37`, `total=10`, `result=147`. El resultado con semilla 2 registrado de forma independiente es 75, no otra interpretación de la misma fuente. Las expectativas de los nuevos archivos de prueba son `3\n1\nnull\n` (`x=2`, `y=3`, `count=1`) y `3\ndivision by zero\n11\n` (`result=3`, `effects=11`). Son resultados del oráculo acotado, no salida de la CLI.
- `node --test web/*.test.cjs`: 39 aprobados, cero fallidos u omitidos. Son pruebas de VM/DOM, no una sesión real de navegador.
- La relectura del Markdown, las comprobaciones locales de enlaces, anclas y rutas de comandos, la preservación de hashes de producción, entradas y PDF, y `git diff --check` forman la compuerta de aceptación de la documentación. Para esta actualización no se ejecutó ninguna sesión de servidor, curl o navegador, ni la suite completa, la compilación completa, vet o la generación de ANTLR.

Auditoría de contribuciones

Comandos de auditoría de solo lectura: `git log d70abf1..633aad1 --format='%h %ad %an %s' --date=short` y `git log --format='%h %ad %an %s' --date=short -- internal/compiscript cmd/compiscript-tac cmd/ide web docs/semestre2/entrega2`.

Los diez commits del primer rango registran el nombre de autor `jonialen` y la fecha **2026-10-18**: `d592b9e` closures; `3a41adb` objetos; `208d709` corrección de campos nominales;

`c242e03` listas/foreach; `40d6cf9` reporte de codegen; `2d2fb4e` compilación en el IDE; `355804c` rechazo atómico de ciclos mal formados; `f416fa2` informe inicial; `9ce55ce` valores/efectos; `633aad1` valores dinámicos protegidos/manejadores. Son nueve commits de implementación más el commit inicial de documentación, no diez contribuciones individuales nuevas certificadas. Los fundamentos anteriores del TAC están registrados entre el 17 y el 18 de octubre con el mismo nombre de autor. El historial de rutas también registra trabajo anterior en el IDE bajo `DijanU` (`e8c9324`, `cc26b6f`, `3cf44ec` el 24 de mayo, y `9ed0d12` el 6 de septiembre). Esos commits anteriores no son evidencia de la contribución de una segunda persona a las unidades recientes de la Entrega 2.

Un registro de autor en Git no certifica qué persona realizó cada tarea. No se infiere ningún porcentaje por integrante, autoría compartida, equivalencia de identidades ni contribución individual. Los integrantes del equipo deben verificar sus declaraciones de contribución reales y el requisito de commits individuales del enunciado antes de la entrega. No se reescribió el historial ni se fabricó ninguna afirmación de autoría.

Brechas de requisitos pendientes y controles de presentación

Brecha histórica resuelta; no se establece ninguna brecha normativa concreta adicional de implementación. La página 1 dice “traducen cada construcción de Compiscript”. La brecha inicial de T8 era real y dio lugar a S11/T9/T10, no a debilitar este requisito. T9 aporta print, residuo, valores de ternarios perezosos y de asignación, almacenamiento de undefined/null y llamadas con valor null; T10 aporta valores desconocidos protegidos, parámetros formales sin tipo y try/catch síncrono. Las pruebas públicas del núcleo y de la CLI construida cubren ahora los ejemplos que antes se rechazaban. La verificación independiente, la corrección y reverificación de cinco casos del contrato de campos y el inventario de fuentes respaldan el despacho de cada tipo de construcción válido del AST. `BadStmt`/`BadExpr` siguen siendo errores de recuperación, no construcciones válidas faltantes; el más unario y los imports no son construcciones de la gramática.

Los errores sintácticos y semánticos siguen devolviendo TAC nil y sin codegen. Las guardas estáticas conservadoras de inicialización de globales, capturas y switch, así como los fallos de ejecución explícitos de tipo, null, undefined, límites y llamadas, siguen siendo contratos documentados. Los programas válidos para Analyze pero inválidos en tiempo de ejecución pueden generar TAC protegido y fallar en el oráculo sin convertirse en nuevos diagnósticos estáticos. Un inventario de cobertura no prueba todos los programas, la terminación, una calificación completa ni la disponibilidad futura del entorno. No se planean más cambios en el producto sin un nuevo requisito concreto o un fallo reproducido.

Comprobaciones humanas y de evaluación aún pendientes: verificar las contribuciones individuales reales, la visibilidad en GitHub y la disponibilidad de las instrucciones y entregables versionados; ejecutar la batería en el entorno de evaluación; abrir el navegador real y comprobar la carga, edición y guardado de `.cps`, Compile/Analyze, los errores ubicados, las vistas de TAC y símbolos, los reinicios y que el comportamiento de YAPar no haya cambiado. Confirmar la disponibilidad de la CDN, el diseño y la accesibilidad, y el funcionamiento el día de la presentación. La prueba automatizada del DOM no sustituye estos pasos.

No es una brecha explícita del PDF: la disposición física en bytes y el ensamblador, un intérprete de TAC como producto y la recolección de basura son responsabilidades posteriores del backend. Las celdas abstractas y los contratos conservados de entorno léxico y de marco están documentados de forma consistente. Los fallos de ejecución en llamadas dinámicas, campos null/undefined y límites son obligaciones del IR, no promesas de que toda compilación aceptada se ejecute sin fallos. Este informe no incluye video ni exportación a Word. El paquete de cuatro PDF solicitado por separado exporta la documentación existente; no agrega evidencia de implementación.